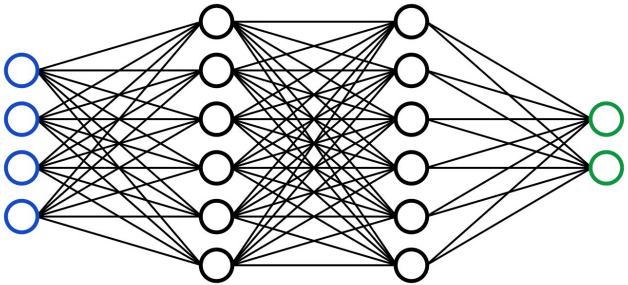
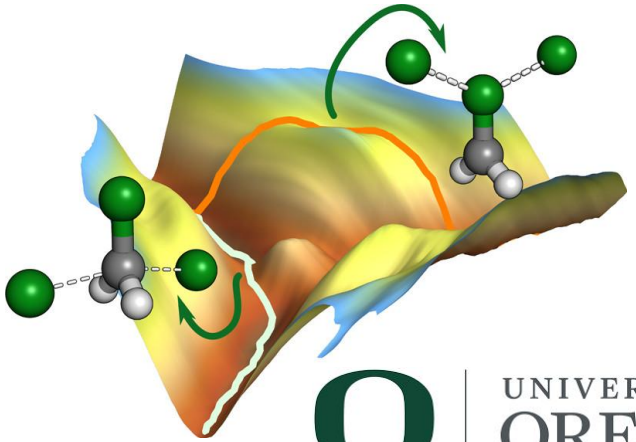
Machine Learning for Chemistry

Winter 2025

Topic 4: Classification





UNIVERSITY OF OREGON

Instructor: Dr. Dhiman Ray

Assistant Professor, Department of Chemistry and Biochemistry

Overview of Topics

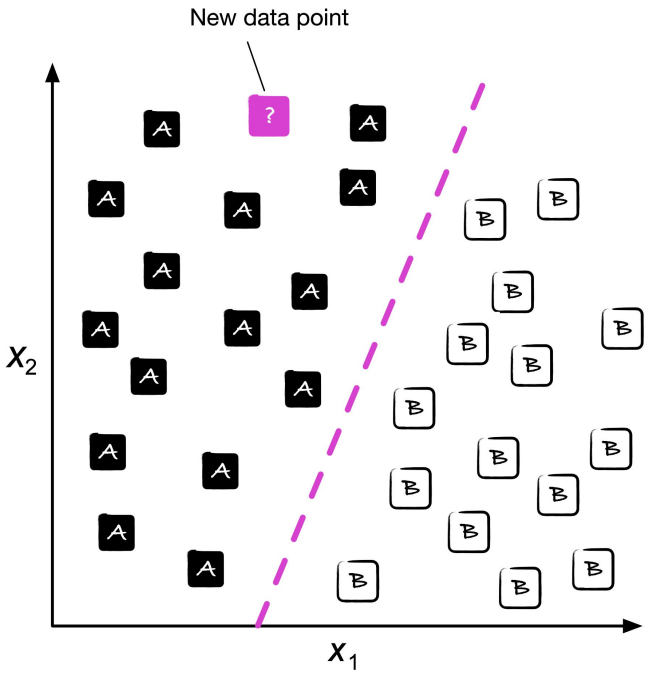
- Logistic Regression
- Softmax Regression
- Support Vector Machine
- k-Nearest Neighbor (kNN) classification
- Assessing Classification Accuracy

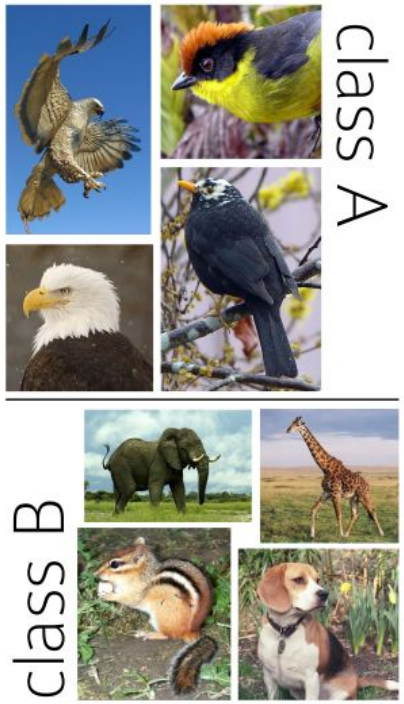
References:

1. **Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow** by Aurélien Géron (Chapter 4 and 5)
2. **Machine Learning with Pytorch and Scikit-Learn** by Raschka, Liu, and Mirjalili (Chapter 3)

Binary Classification

You have many examples that falls in class A and many more for class B. For a new data-point, decide whether it belongs to class A or B.

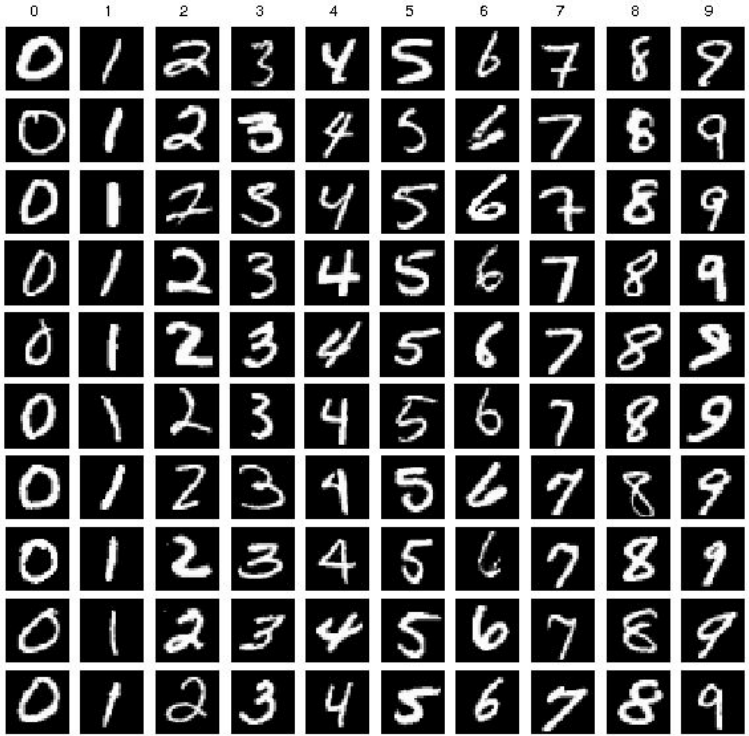


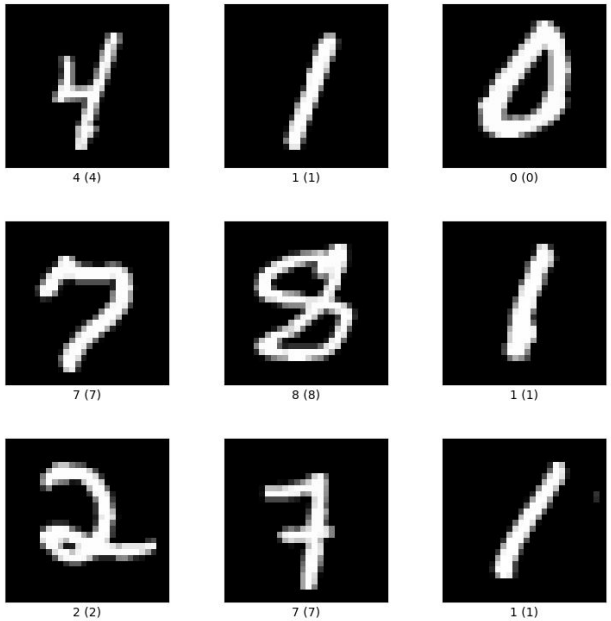


New data

Multiclass Classification

Similar to binary classification but with many classes.





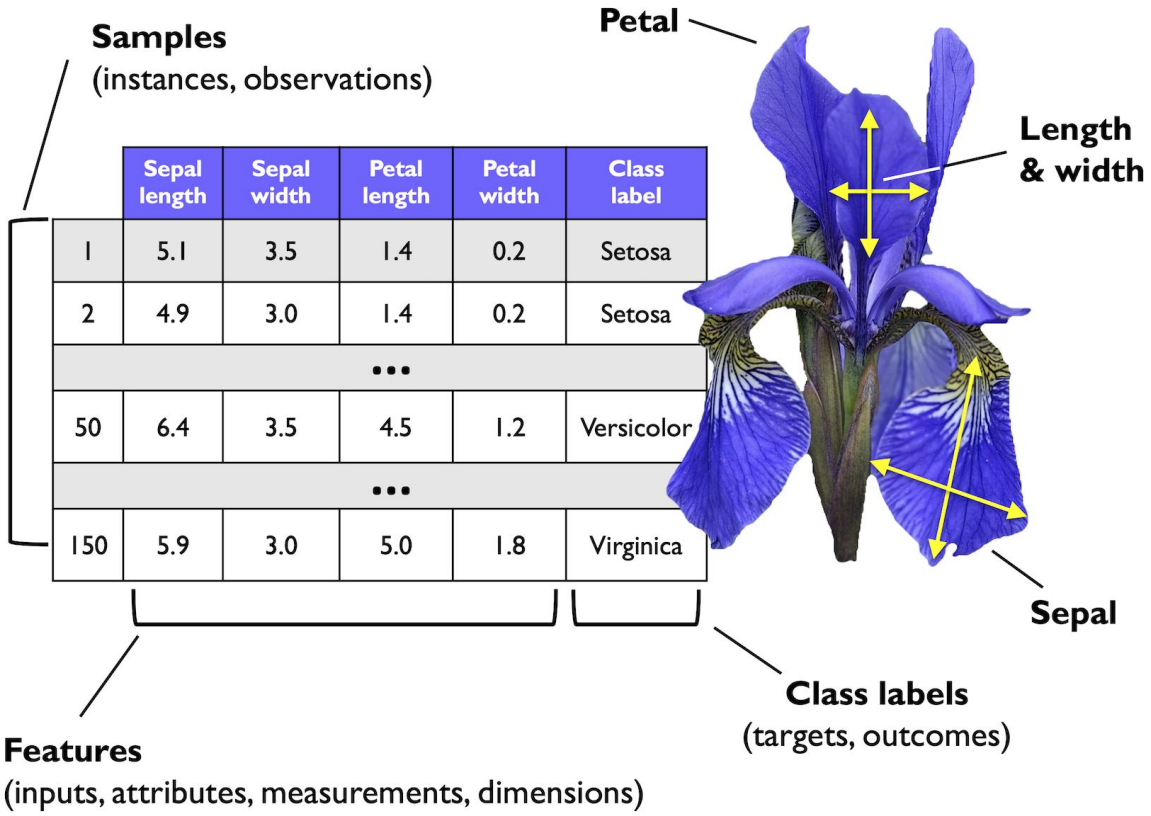
MNIST dataset of handwritten digits

Iris Dataset

The Iris dataset is a classic example for classification in machine learning (https://archive.ics.uci.edu/ml/datasets/iris).

It contains the measurements of 150 Iris flowers from three different species: Setosa, Versicolor, and Virginica.

We will use the Iris dataset to explain classification algorithms.



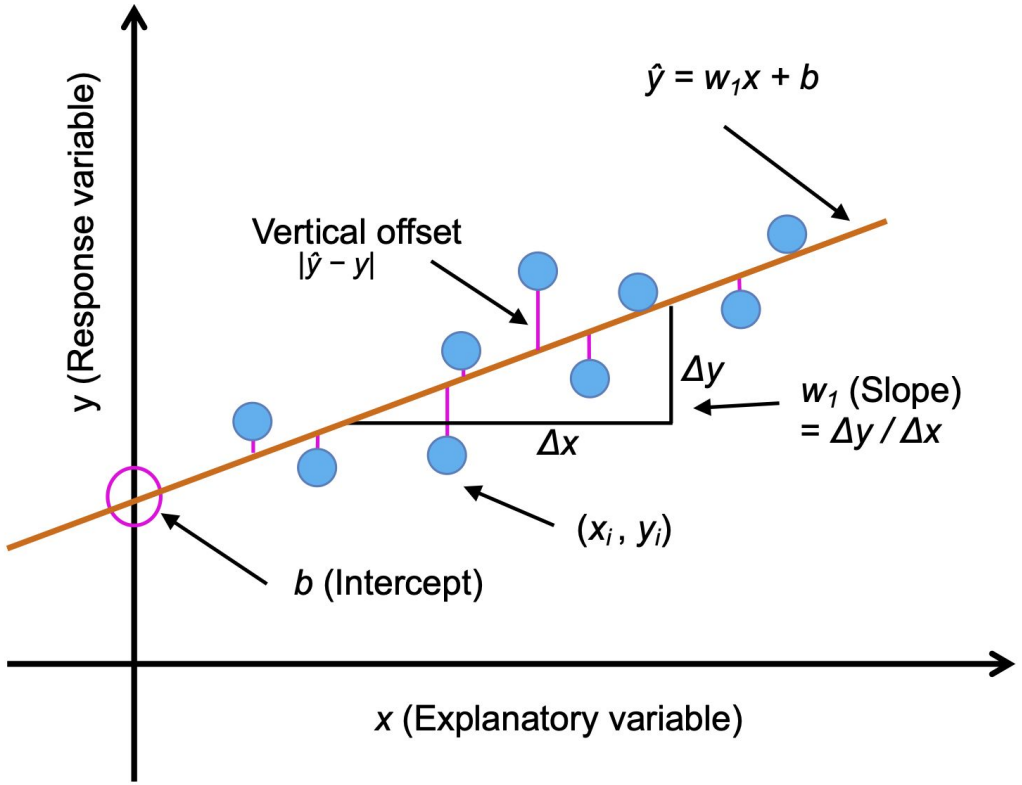
	Sepal length	Sepal width	Petal length	Petal width	Class label
1	5.1	3.5	1.4	0.2	Setosa
2	4.9	3.0	1.4	0.2	Setosa
...					
50	6.4	3.5	4.5	1.2	Versicolor
...					
150	5.9	3.0	5.0	1.8	Virginica

Recap of Linear Regression

Linear regression models the relationship between **features** (explanatory variable, *x*) and a continuous-valued **target** (response variable, *y*).

$$y = w_1 x + b$$

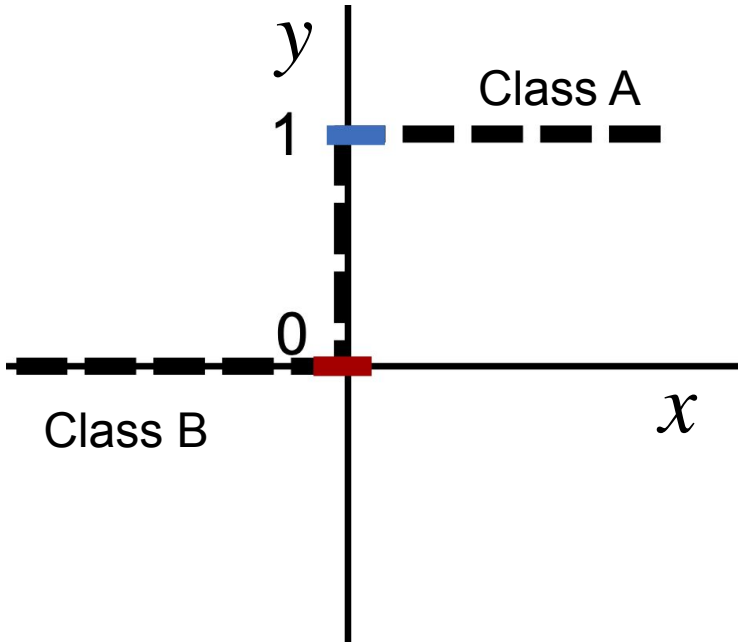
Bias *b*, represents the y axis intercept and w_1 is the **weight** coefficient



Linear Regression is not good for Classification

But classification problems are different!

Say y=1 in class A and y = 0 in class B.



Linear Regression is not good for Classification

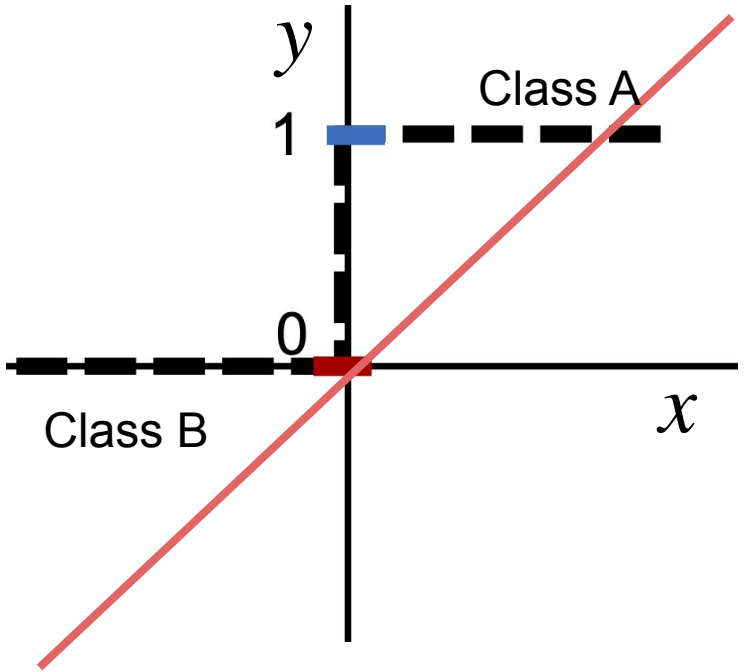
But classification problems are different!

Say y=1 in class A and y = 0 in class B.

It is very difficult to fit such a step function with a straight line.

$$y = w_1 x + b$$

MSE Loss will go to infinity



Logistic Regression

Transform the output of the linear combination with a **logistic** or **sigmoid** function.

Predicted value:

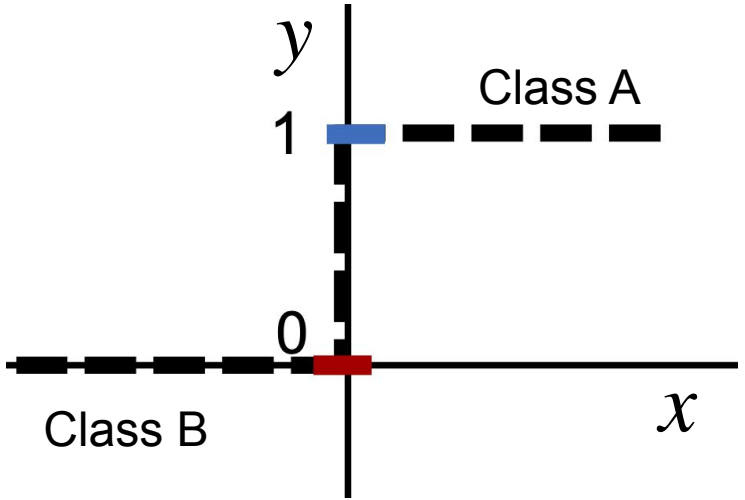
$$\hat{y} = \sigma(w_1 x + b)$$

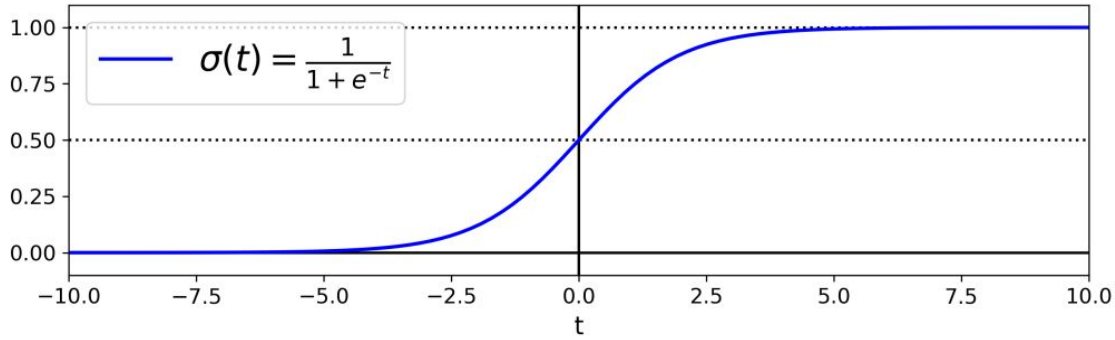
where

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

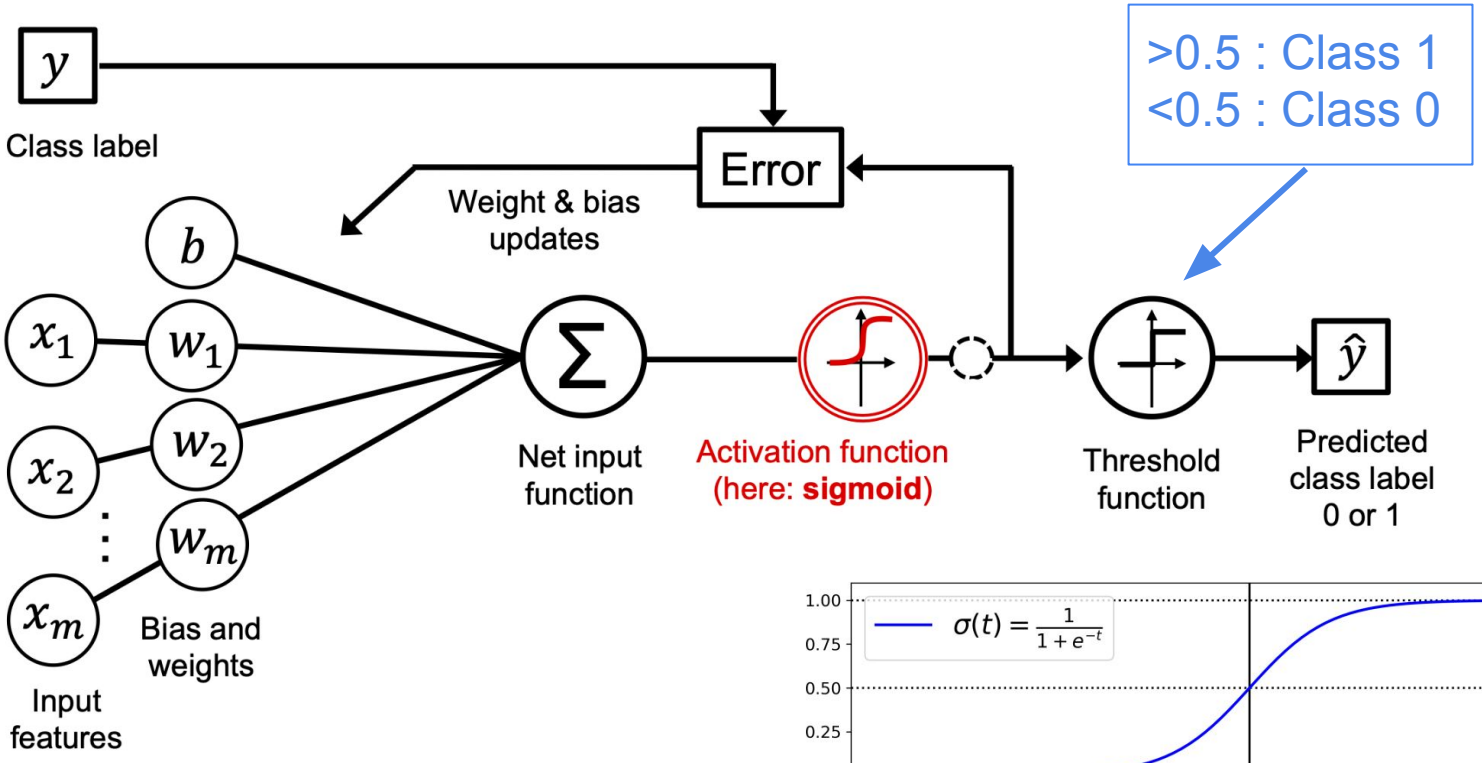
In multidimensional case

$$\hat{y} = \sigma\left(\sum_{i=0}^{n} \theta_i x_i\right)$$





Logistic Regression



Logistic Regression: Weight & Bias Update

It can be shown that, using Gradient Descent, the parameters θ of Logistic regression can be updated as:

$$\theta_k^{(nextstep)} = \theta_k - \eta \cdot \sum_{i=1}^{m} \left(\sigma\left(\sum_{j=0}^{n} \theta_j x_j^{(i)} \right) - y^{(i)} \right) \cdot x_k^{(i)}$$

This is virtually identical to that of linear regression:

$$\theta_k^{(nextstep)} = \theta_k - \eta \cdot \sum_{i=1}^{m} \left(\sum_{j=0}^{n} \theta_j x_j^{(i)} - y^{(i)} \right) \cdot x_k^{(i)}$$

Only difference is the sigmoid operation (activation function)

You can also use stochastic gradient descent and mini-batch gradient descent

Logistic Regression: Weight & Bias Update

It can be shown that, using Gradient Descent, the parameters θ of Logistic regression can be updated as:

$$\theta_k^{(nextstep)} = \theta_k - \eta \cdot \sum_{i=1}^{m} \left(\sigma\left(\sum_{j=0}^{n} \theta_j x_j^{(i)} \right) - y^{(i)} \right) \cdot x_k^{(i)}$$

This is virtually identical to that of linear regression:

$$\theta_k^{(nextstep)} = \theta_k - \eta \cdot \sum_{i=1}^{m} \left(\sum_{j=0}^{n} \theta_j x_j^{(i)} - y^{(i)} \right) \cdot x_k^{(i)}$$

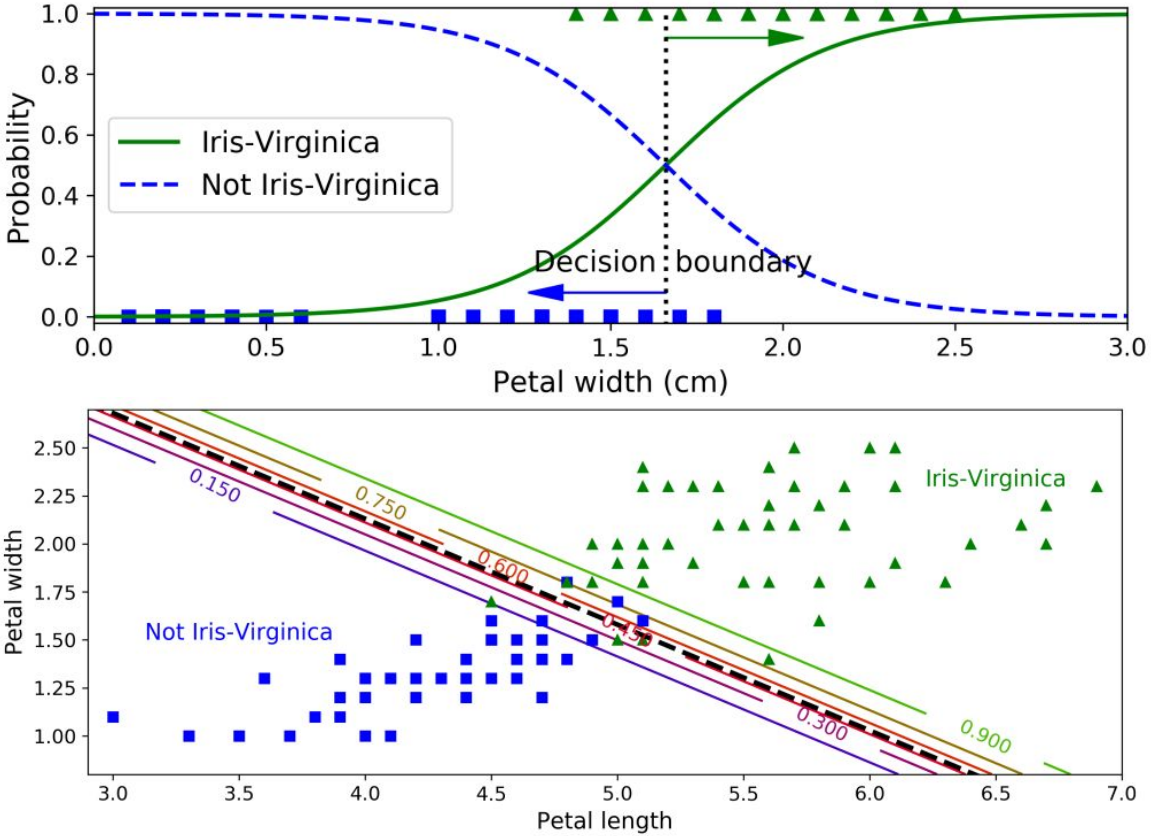
Only difference is the sigmoid operation (activation function)

You can also use stochastic gradient descent and mini-batch gradient descent

Logistic Regression: Example

Classifying the Iris-Virginica species flower from the rest using 1D or 2D feature space.

Decision boundary: black dashed line which separates the two classes in the feature space.



Multiclass Classification

Training Classes

Multiclass Classification

Training Classes

New Data

One-hot-encoding for Class Labels

Training Classes

Class labels:

Dog = 0, Cat = 1, Boat = 2, Airplane = 3

One-hot-encoding for Class Labels

Training Classes

Class labels:

Dog = 0, Cat = 1, Boat = 2, Airplane = 3

No! This leads to artificial ordering. The actual classes have no ordering.

One-hot-encoding for Class Labels

Training Classes

Class labels:

Dog = 0, Cat = 1, Boat = 2, Airplane = 3

No! This leads to artificial ordering. The actual classes have no ordering.

One hot encoding labels:

Dog = [1, 0, 0, 0]

Cat = [0, 1, 0, 0]

Boat = [0, 0, 1, 0]

Airplane = [0, 0, 0, 1]

Increase the label dimension to be equal to the number of classes.

Multiclass Classification: SoftMax Regression

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n = \boldsymbol{\theta}^{\mathbf{T}} \mathbf{x},$$

This linear combination now predicts the probability of being in class k.

Multiclass Classification: SoftMax Regression

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n = \boldsymbol{\theta}^{\mathrm{T}} \mathbf{x},$$

This linear combination now predicts the probability of being in class k.

Softmax score: $s_k(\mathbf{x}) = \mathbf{x}^T \boldsymbol{\theta}^{(k)}$

where $\theta^{(k)}$ is the dedicated parameter set for class k.

Multiclass Classification: SoftMax Regression

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n = \boldsymbol{\theta}^{\mathrm{T}} \mathbf{x},$$

This linear combination now predicts the probability of being in class ***k***.

Softmax score: $s_k(\mathbf{x}) = \mathbf{x}^T \boldsymbol{\theta}^{(k)}$

where $\theta^{(k)}$ is the dedicated parameter set for class ***k***.

Normalized probability: $\hat{p}_k = \sigma(\mathbf{s}(\mathbf{x}))_k = \dfrac{\exp\left(s_k(\mathbf{x})\right)}{\sum_{j=1}^{K} \exp\left(s_j(\mathbf{x})\right)}$

K is the number of classes.

Multiclass Classification: SoftMax Regression

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n = \boldsymbol{\theta}^{\mathbf{T}} \mathbf{x},$$

This linear combination now predicts the probability of being in class k.

Softmax score: $s_k(\mathbf{x}) = \mathbf{x}^T \boldsymbol{\theta}^{(k)}$

where $\theta^{(k)}$ is the dedicated parameter set for class k.

Normalized probability: $\hat{p}_k = \sigma(\mathbf{s}(\mathbf{x}))_k = \dfrac{\exp\left(s_k(\mathbf{x})\right)}{\sum_{j=1}^{K} \exp\left(s_j(\mathbf{x})\right)}$

K is the number of classes.

Notice the similarity with partition function in statistical mechanics!

Multiclass Classification: SoftMax Regression

Cross Entropy Loss Function: $J(\Theta) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K} y_k^{(i)} \log\left(\hat{p}_k^{(i)}\right)$

It's gradient: $\nabla_{\theta^{(k)}} J(\Theta) = \frac{1}{m}\sum_{i=1}^{m}\left(\hat{p}_k^{(i)} - y_k^{(i)}\right)\mathbf{x}^{(i)}$ can be used in gradient descent.

Here $y_k^{(i)}$ = 1 if the data belongs to class k and zero otherwise.

Notice the similarity with Linear and Logistic Regression.

All these update rules use efficient linear algebra libraries to speed up calculation.

Multiclass Classification: SoftMax Regression

Predicted class: $\hat{y} = \underset{k}{\text{argmax}}\ \sigma(\mathbf{s}(\mathbf{x}))_k$

	Scoring Function	Unnormalized Probabilities	Normalized Probabilities
Dog	-3.44	0.0321	0.0006
Cat	1.16	3.1899	0.0596
Boat	-0.81	0.4449	0.0083
Airplane	3.91	49.8990	0.9315

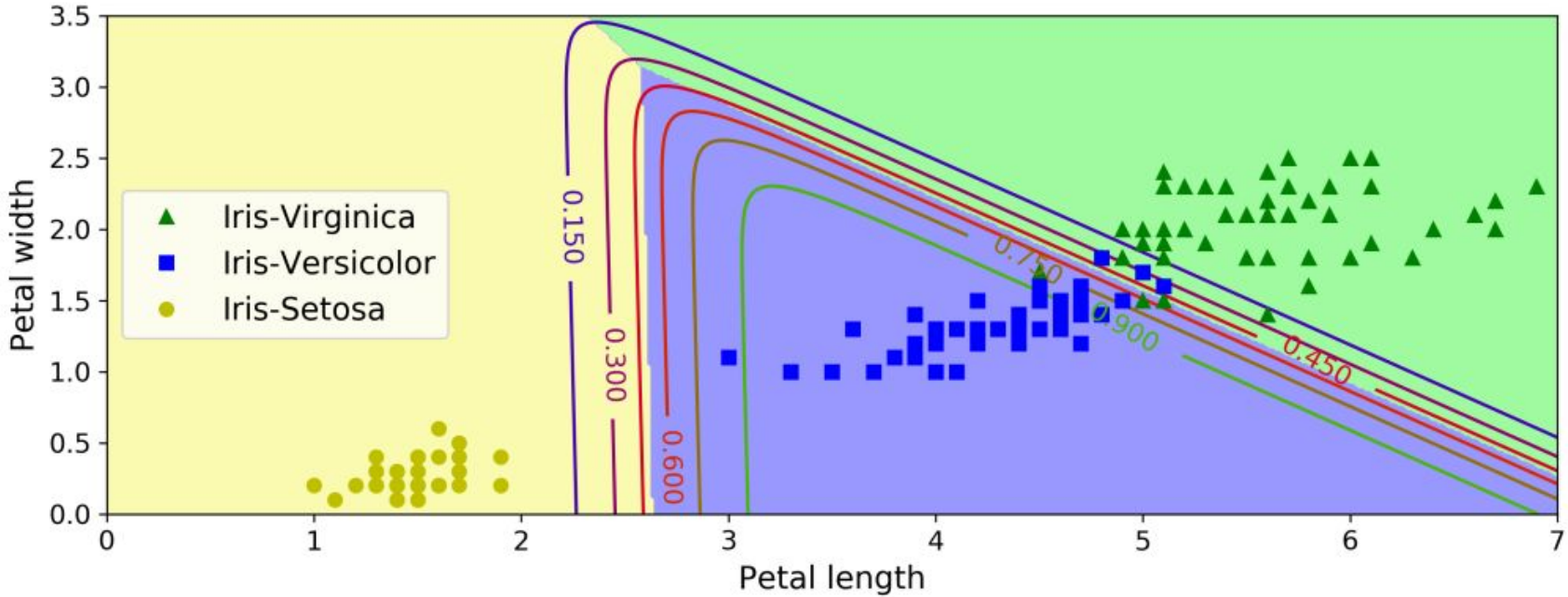
Multiclass Classification: SoftMax Regression

Predicted class: $\hat{y} = \underset{k}{\text{argmax}}\ \sigma(\mathbf{s}(\mathbf{x}))_k$

	Scoring Function	Unnormalized Probabilities	Normalized Probabilities
Dog	-3.44	0.0321	0.0006
Cat	1.16	3.1899	0.0596
Boat	-0.81	0.4449	0.0083
Airplane	3.91	49.8990	0.9315

Matches the one hot encoding class label of airplane: [0, 0, 0, 1]

SoftMax Regression on Iris Dataset



All points in each different color corresponds to different class

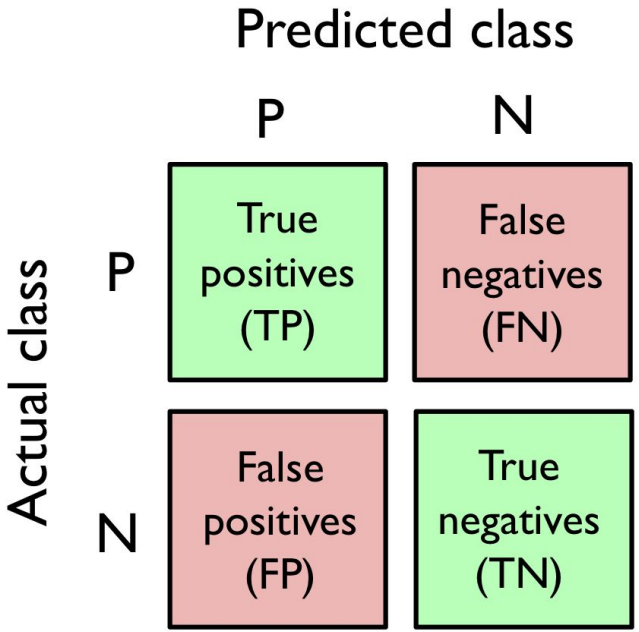
Assessing Classification Accuracy

We will discuss the following methods to assess classification accuracy

1. Confusion Matrix
2. Precision-Recall Curve
3. Receiver Operating Characteristic (ROC) Curve

Confusion Matrix

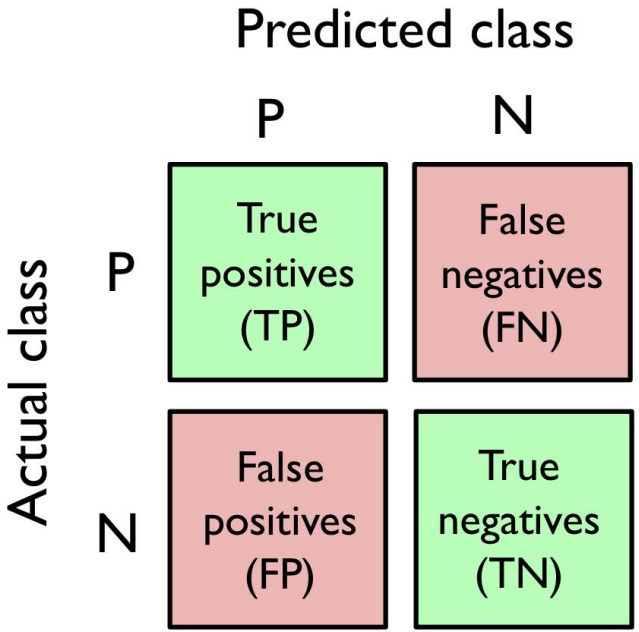
Actual	A	A	B	A	B	B	A	A	B	A	B	B	B	A	A	B	A	A
Predicted	A	A	A	A	B	B	A	A	A	B	B	A	B	B	A	B	A	A



	A	B
A		
B		

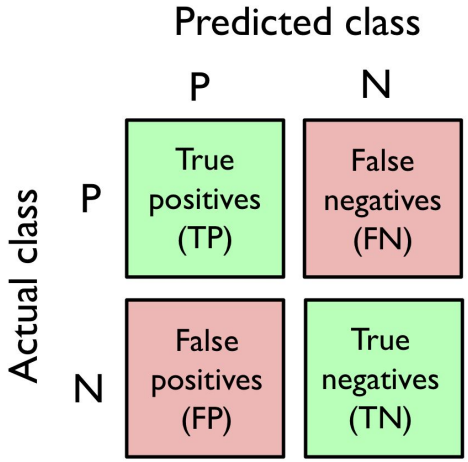
Confusion Matrix

Actual	A	A	B	A	B	B	A	A	B	A	B	B	B	A	A	B	A	A
Predicted	A	A	A	A	B	B	A	A	A	B	B	A	B	B	A	B	A	A



	A	B
A	8	2
B	3	5

Confusion Matrix



	A	B
A	8	2
B	3	5

Precision (PRE) = $\frac{TP}{TP + FP}$ = 8/(8+3) = 0.73

Recall (REC) = $\frac{TP}{FN + TP}$ = 8/(2+8) = 0.8

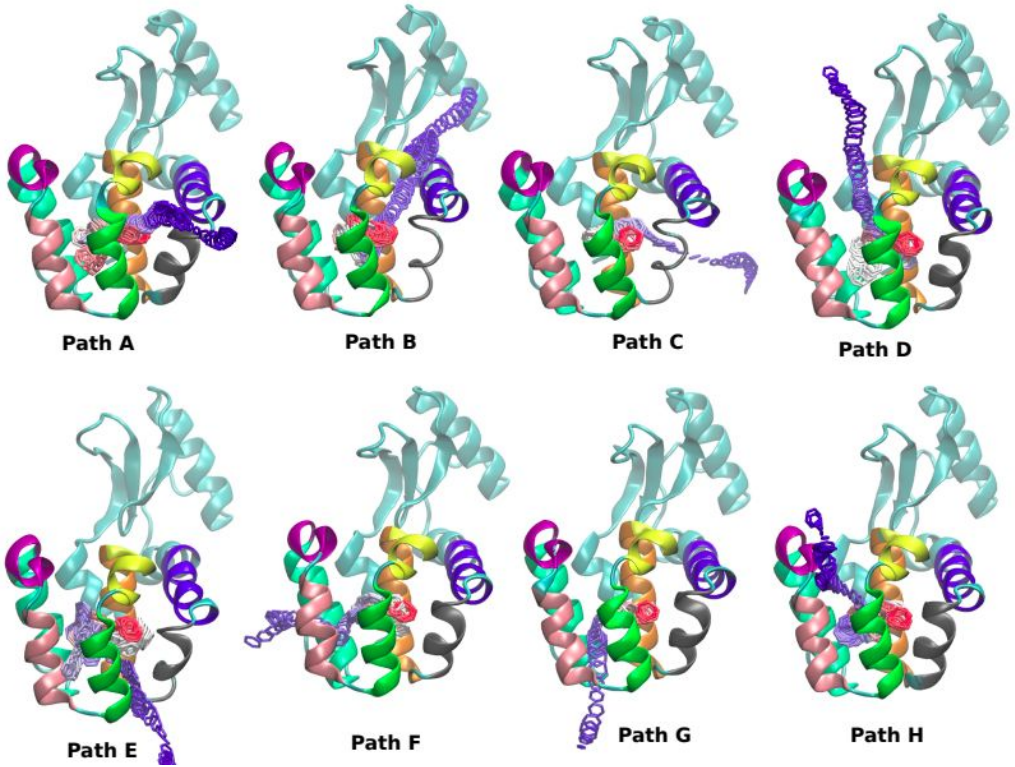
F1-score = $2\frac{PRE \times REC}{PRE + REC}$

= 2 x (0.73 x 0.8)/(0.73 + 0.8)

= 0.76

Overall accuracy is 76%

Example: Classification of Drug Binding Pathways



Manual Classification

DTW_D Classification	A	B	C	D	E	F	G	H
A	8		1					
B								
C	1	2	25					
D	1			18				2
D2				1				
E					3			
F+G						7	13	2
H				4				12

Class	Precision	Recall	F1 score
A	0.89	0.8	0.84
B	-	0	-
C	0.89	0.96	0.92
D	0.85	0.78	0.81
E	1.00	1.00	1
F+G	0.91	1.00	0.95
H	0.75	0.75	0.75

Ray and Parrinello. PNAS 2024 https://doi.org/10.1073/pnas.2313542121

Precision Recall Curve

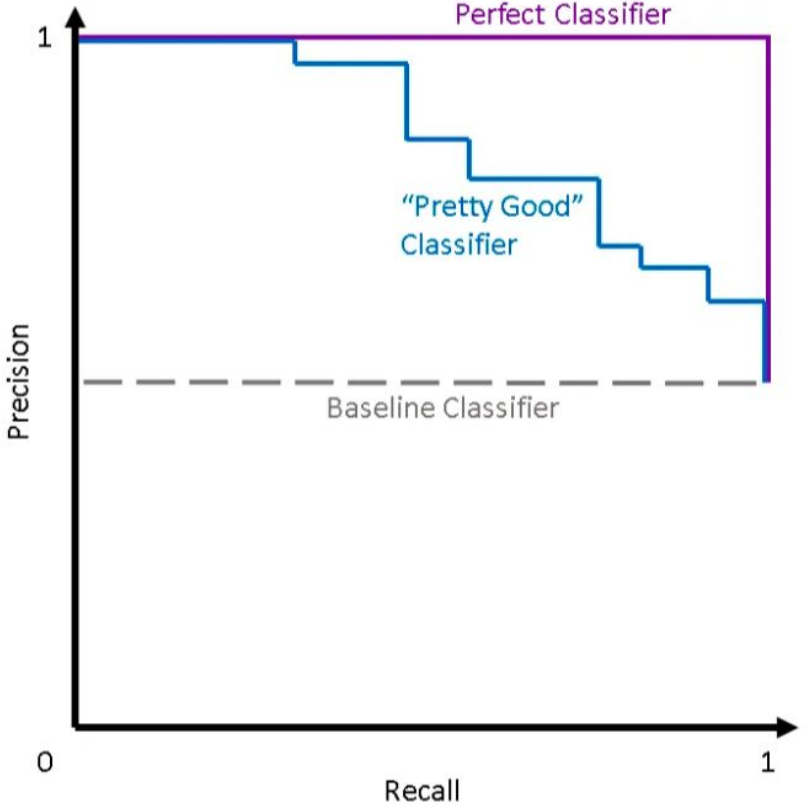
A graph of precision vs recall for a given classification model.

The baseline model predicts “positive” all the time: it’s precision is always 0.5

The perfect classifier always makes the correct prediction: it’s precision is always 1

A real-life classifier that has some predictive value falls in-between.

Area under the Curve (AUC) is a measure of classification accuracy

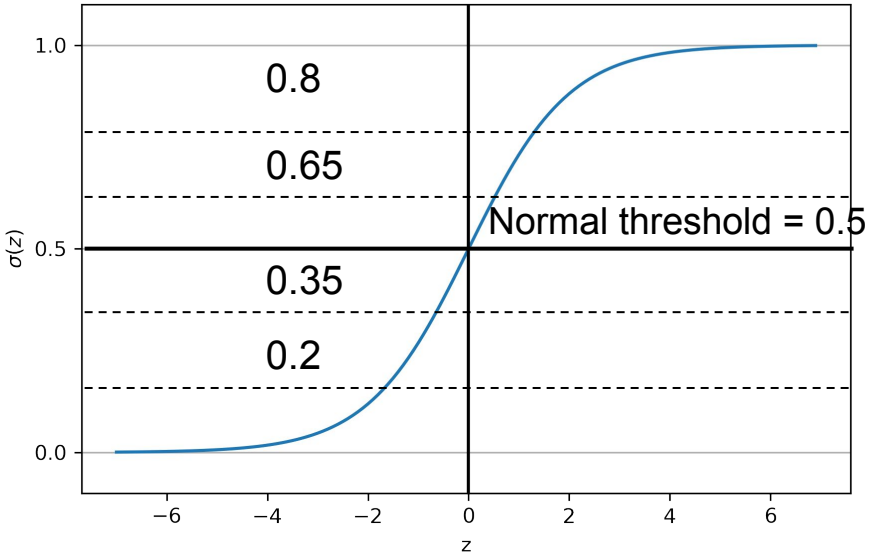


Precision Recall Curve

To construct a precision-recall curve we need to obtain the precision and recall values for the same classifier with different prediction threshold.

Normally when $\sigma(z) > 0.5$ we predict “positive” and when $\sigma(z) < 0.5$ we predict “negative”.

Instead we need to move the threshold between 0 and 1 and compute the precision and recall for all situations keeping the model parameters unchanged.



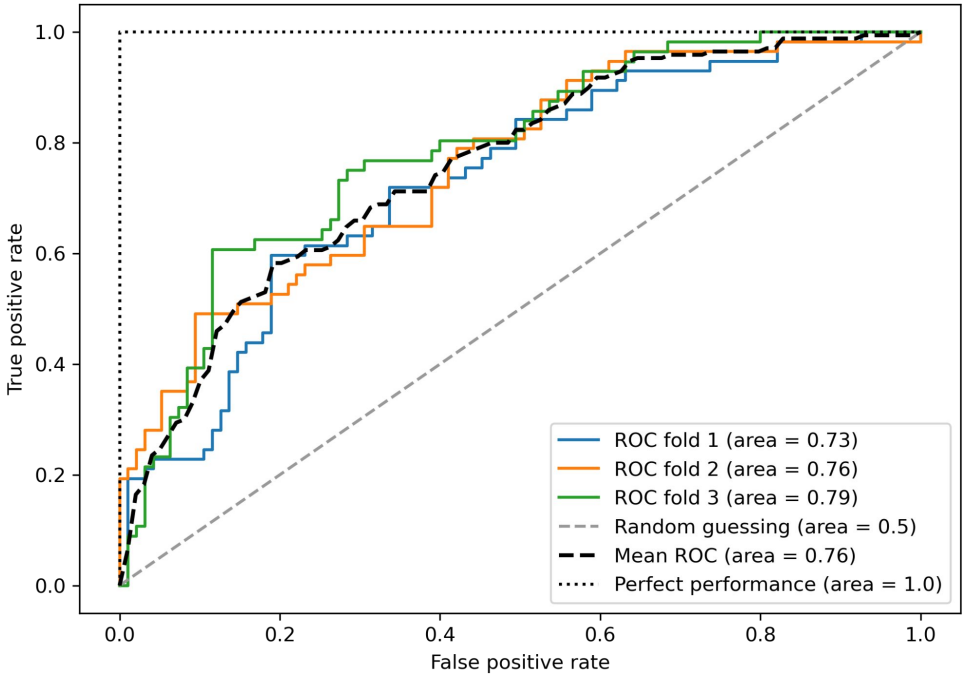
Receiver Operating Characteristic (ROC) Curve

A graph of the True Positive rate vs. False Positive rate for different thresholds for the same model.

The diagonal line is random guess. Area under the curve (AUC) = 0.5

Perfect performance has area 1.0

A real-life classifier that has some predictive value falls in-between.

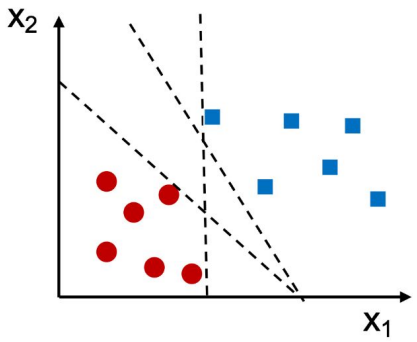


Jupyter Notebook Tutorial

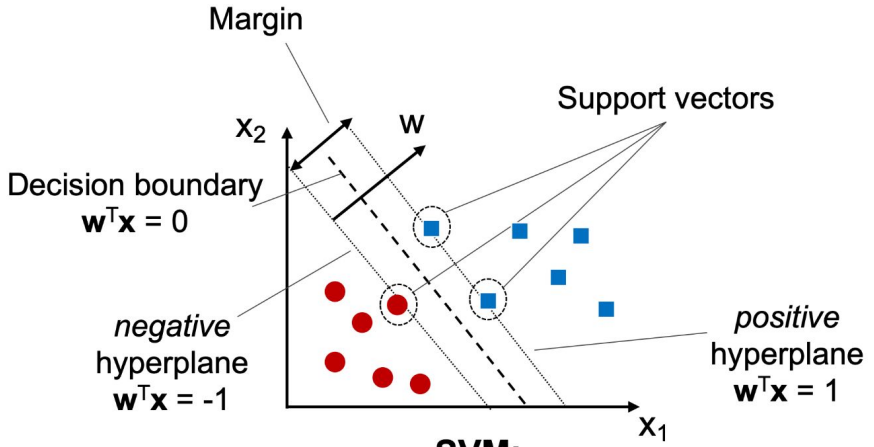
Support Vector Machine (SVM)

SVM is one of the most popular ML algorithm similar to neural networks.

The objective of SVM is to maximize the the distance between the separating hyperplane (**decision boundary**) and the training examples that are closest to this hyperplane (**support vectors**)



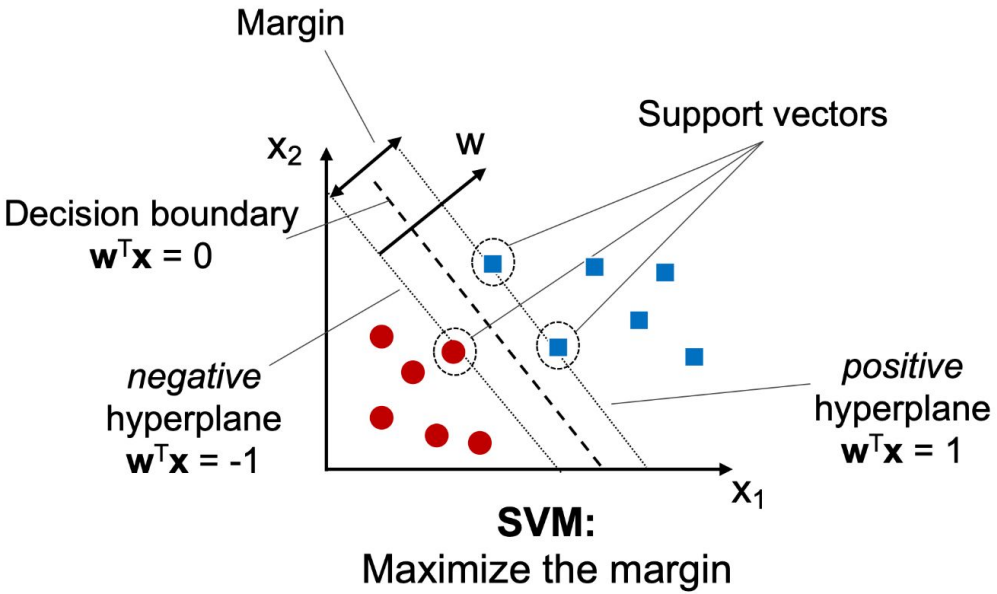
Which hyperplane?



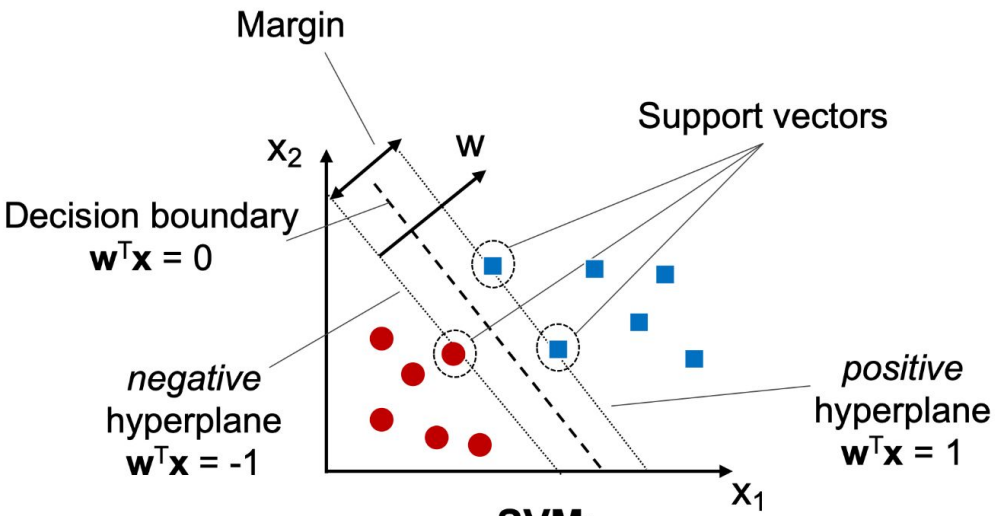
SVM:
Maximize the margin

Support Vector Machine (SVM)

Decision boundary: $w_1 x_1 + w_2 x_2 = 0$



Support Vector Machine (SVM)

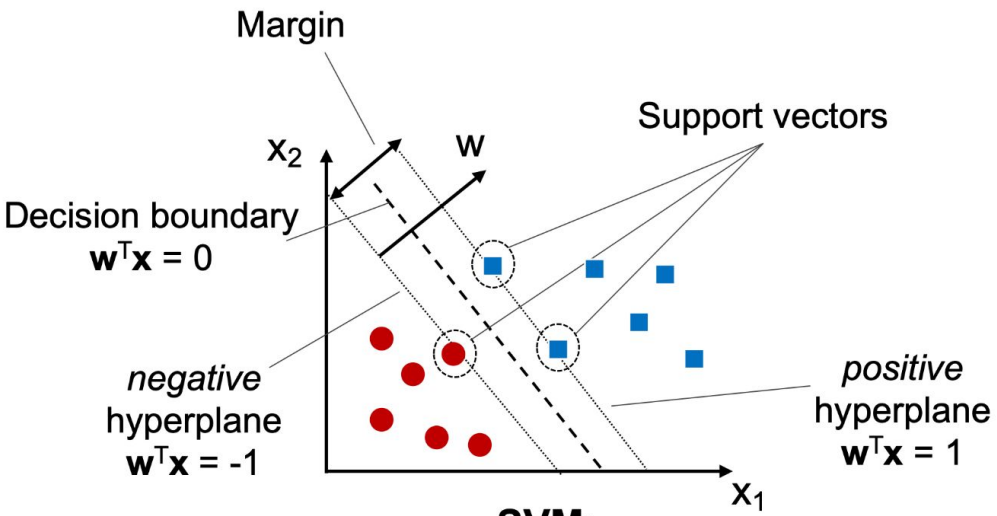


SVM:
Maximize the margin

Decision boundary: $w_1x_1 + w_2x_2 = 0$

Negative/Positive Hyperplanes:
Planes parallel to decision boundary and passing through the point closest to the decision boundary. $w_1x_1 + w_2x_2 = \pm 1$

Support Vector Machine (SVM)

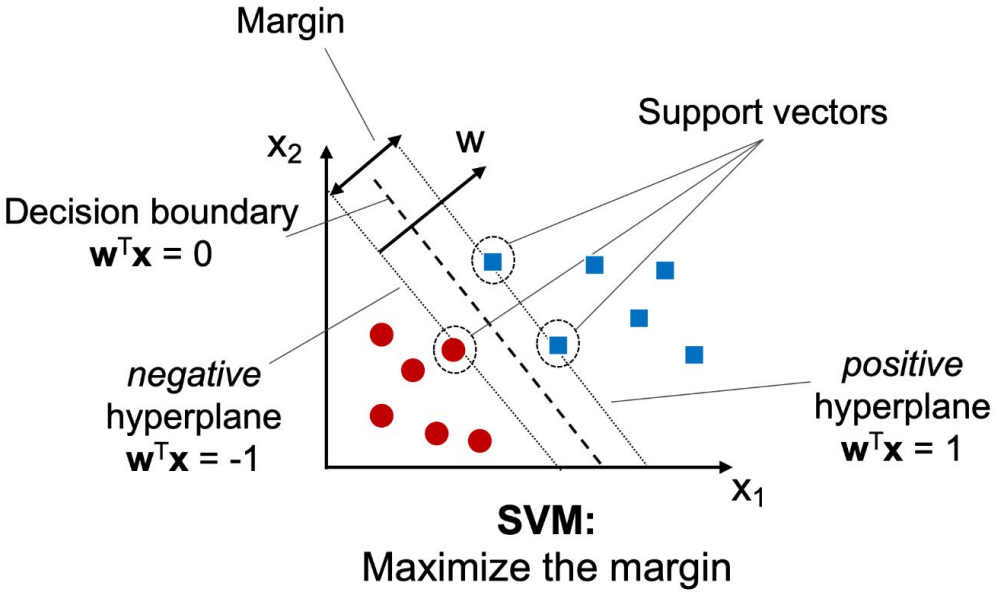


Decision boundary: $w_1x_1 + w_2x_2 = 0$

Negative Hyperplane: Plane parallel to decision boundary and passing through the point closest to the decision boundary. $w_1x_1 + w_2x_2 = \pm 1$

Margin: width of the road separating two classes is $\frac{2}{||w||}$ that needs to be maximized.

Support Vector Machine (SVM)



Decision boundary: $w_1x_1 + w_2x_2 = 0$

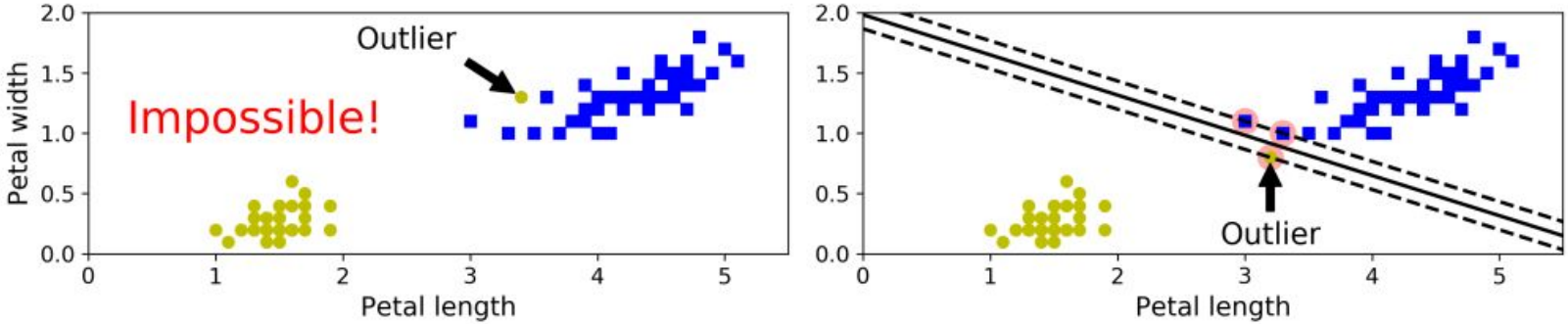
Negative Hyperplane: Plane parallel to decision boundary and passing through the point closest to the decision boundary. $w_1x_1 + w_2x_2 = \pm 1$

Margin: width of the road separating two classes is $\frac{2}{||w||}$ that needs to be maximized.

Loss function: $\mathcal{L}_{SVM} = \frac{1}{2}||w||^2$

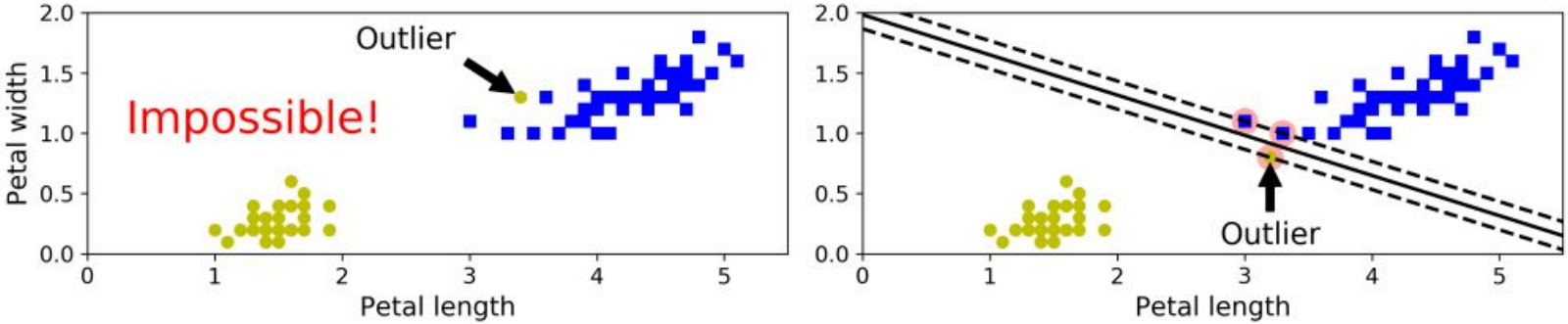
Support Vector Machine (SVM)

Standard or Hard margin SVMs are highly sensitive to outliers and feature scaling. Here is an example from the Iris dataset.



Support Vector Machine (SVM)

Standard or Hard margin SVMs are highly sensitive to outliers and feature scaling. Here is an example from the Iris dataset.



A slack parameter (C) is used for regularizing SVMs.

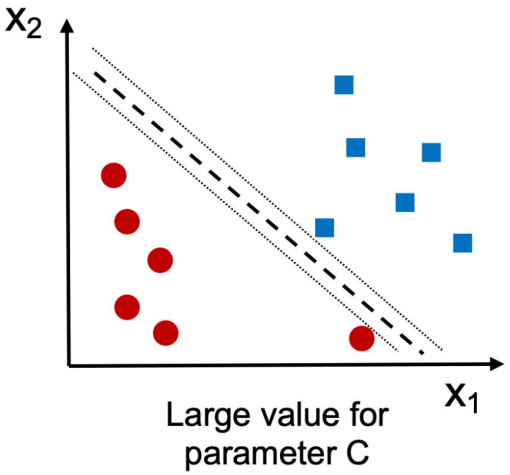
Loss function of Soft margin SVM: $\mathcal{L}_{SVM}^{\text{soft}} = \frac{1}{2}||w||^2 + C\sum_i \xi_i$

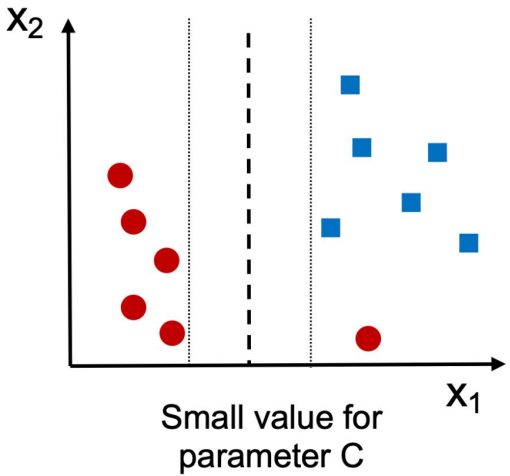
where ξ is a measure of misclassified samples

Support Vector Machine (SVM)

Slack Parameter (C): Hyperparameter for controlling the penalty for misclassification.

$$\mathcal{L}_{SVM}^{\text{soft}} = \frac{1}{2}||w||^2 + C\sum_i \xi_i$$



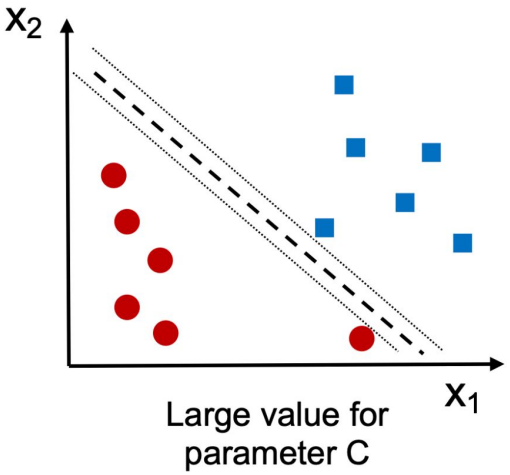


Support Vector Machine (SVM)

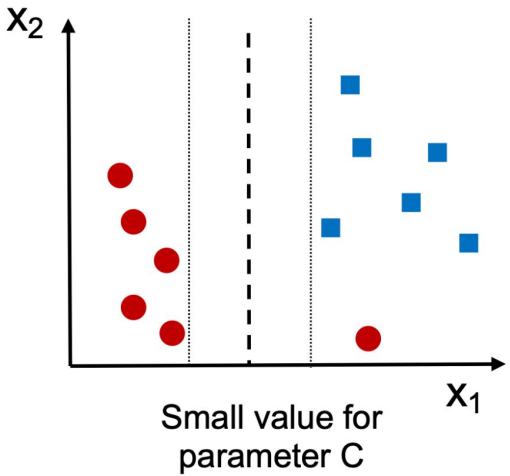
Slack Parameter (C): Hyperparameter for controlling the penalty for misclassification.

$$\mathcal{L}_{SVM}^{\text{soft}} = \frac{1}{2}||w||^2 + C\sum_i \xi_i$$

Large Slack (C) allows only small values of ξ. i.e. little to no misclassification is allowed. **(Low bias high variance i.e. Overfitting)**



Large value for parameter C

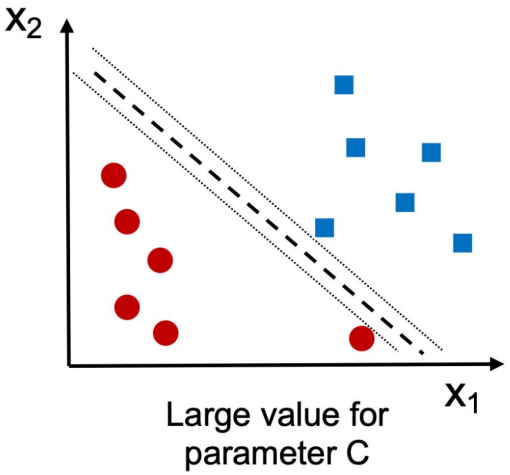


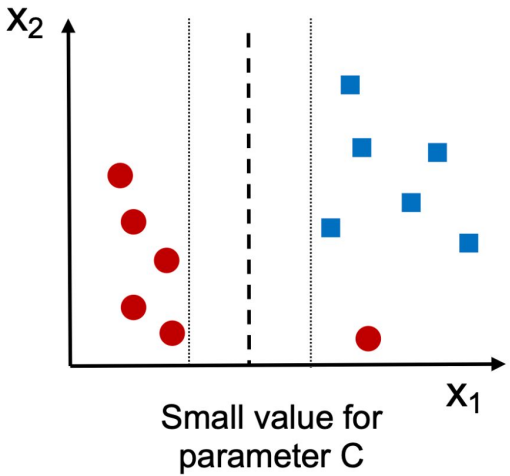
Small value for parameter C

Support Vector Machine (SVM)

Slack Parameter (C): Hyperparameter for controlling the penalty for misclassification.

$$\mathcal{L}_{SVM}^{\text{soft}} = \frac{1}{2}||w||^2 + C\sum_i \xi_i$$



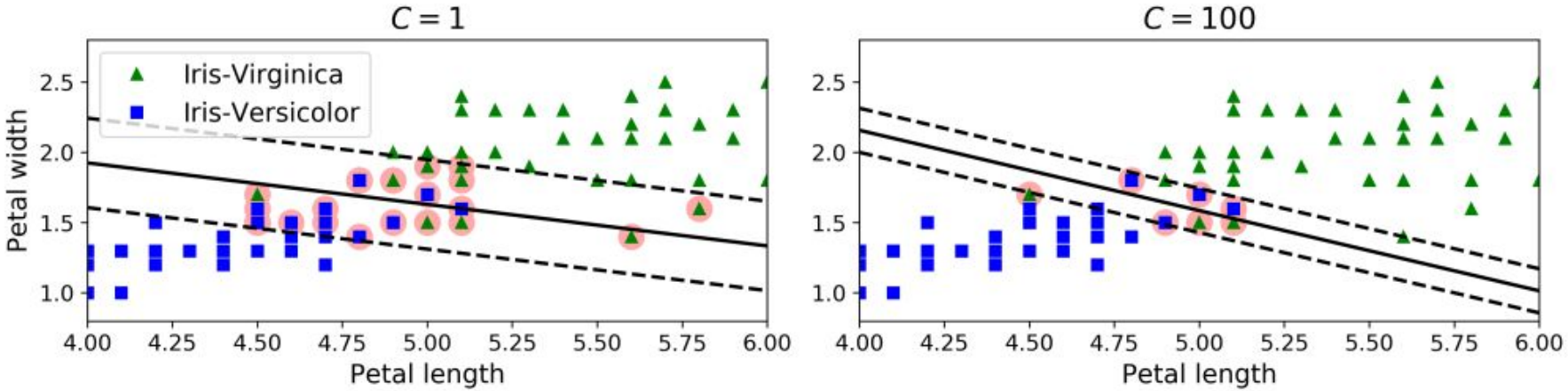


Large Slack (C) allows only small values of ξ. i.e. little to no misclassification is allowed. **(Low bias high variance i.e. Overfitting)**

Small Slack (C) means higher values of ξ are also allowed when minimizing the loss function i.e. more misclassification is allowed. **(High bias low variance i.e. Underfitting)**

Support Vector Machine (SVM)

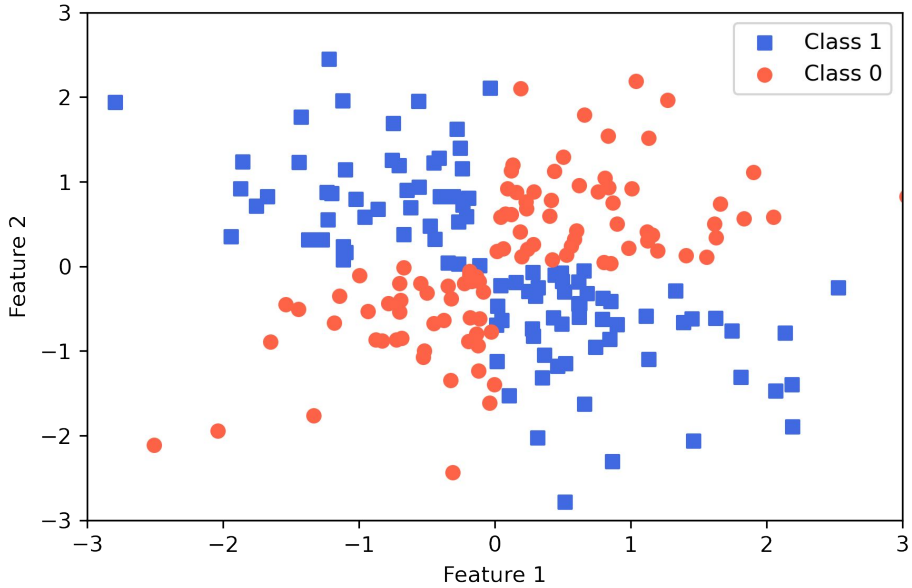
Example: Classification of the Iris Dataset.



Non-linear Support Vector Machine

Not all data is linearly separable!

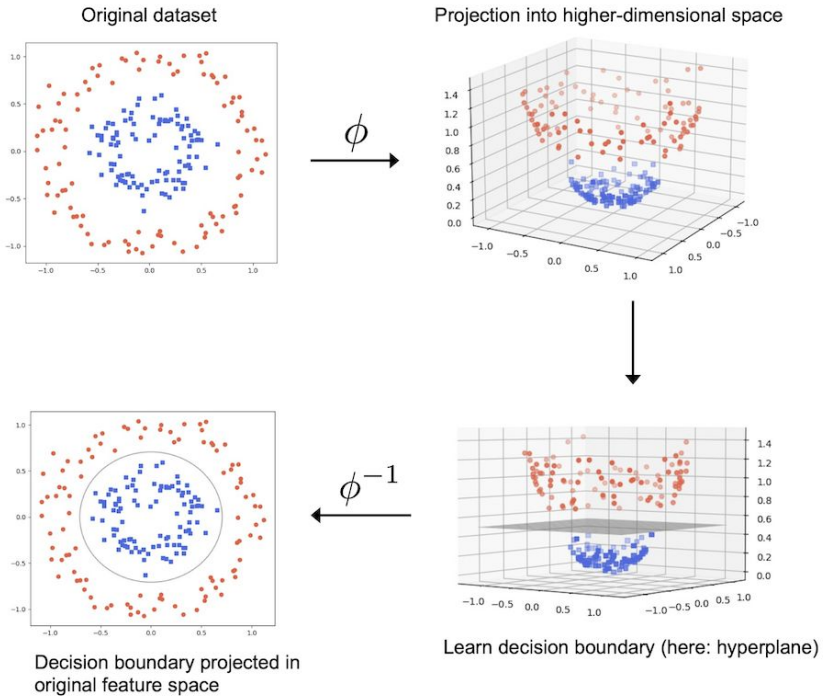
For example, see the XOR dataset. Even in this apparently simple 2D dataset cannot be separated with linear decision boundary.



Non-linear Support Vector Machine

One way to deal with such linearly inseparable data by creating nonlinear combinations of the original features to project them onto a higher-dimensional space via a mapping function, ϕ, where the data becomes linearly separable.

$$\phi(x_1, x_2) = (z_1, z_2, z_3) = (x_1, x_2, x_1^2 + x_2^2)$$

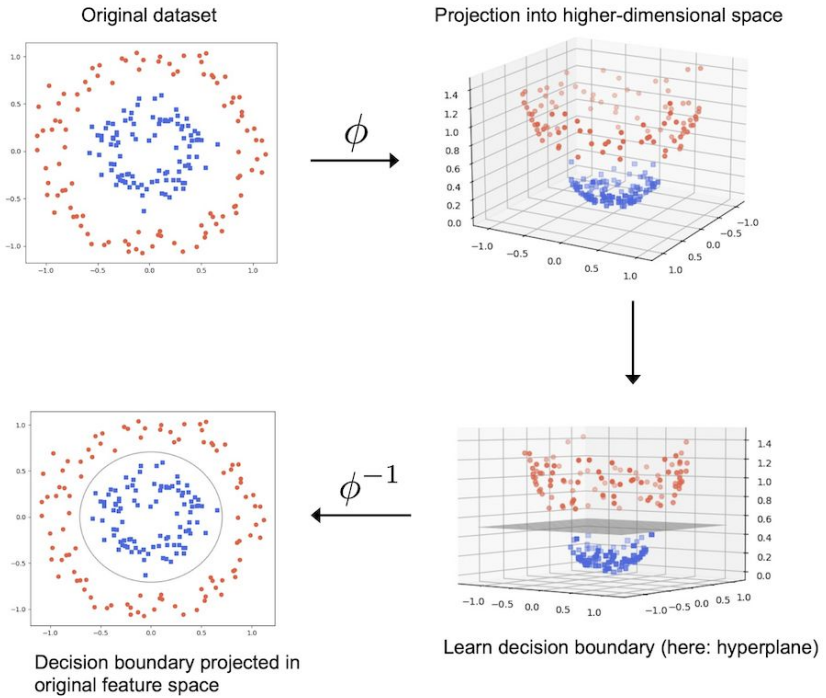


Non-linear Support Vector Machine

One way to deal with such linearly inseparable data by creating nonlinear combinations of the original features to project them onto a higher-dimensional space via a mapping function, ϕ, where the data becomes linearly separable.

$$\phi(x_1, x_2) = (z_1, z_2, z_3) = (x_1, x_2, x_1^2 + x_2^2)$$

But do you see any problem?



Problem of Polynomial Expansion

$$\phi(x_1, x_2) \rightarrow x_1, \quad x_2, \quad f(x_1, x_2)$$

$$\phi(x_1, x_2, x_3) \rightarrow x_1, \quad x_2, \quad x_3,$$
$$f(x_1, x_2), \quad f(x_1, x_3), \quad f(x_2, x_3),$$
$$g(x_1, x_2, x_3)$$

$$\phi(x_1, x_2, x_3, x_4) \rightarrow x_1, \quad x_2, \quad x_3, \quad x_4,$$
$$f(x_1, x_2), \quad f(x_1, x_3), \quad f(x_2, x_3),$$
$$f(x_1, x_4), \quad f(x_2, x_4), \quad f(x_3, x_4),$$
$$g(x_1, x_2, x_3), \; g(x_1, x_2, x_4), \; g(x_2, x_3, x_4), \; g(x_1, x_3, x_4),$$
$$h(x_1, x_2, x_3, x_4)$$

With increasing number of features, the dimensionality of the feature vector is exploding!

Use **Kernel Trick**

Kernel Trick in Kernel-SVM

For SVMs, the decision boundary is calculated by computing dot products between data-point vectors in the input space. Computing dot products between kernel functions can get expensive when dealing with high-dimensional data.

Kernel trick replaces the expensive dot product with a kernel function.

Equation 5-10. Common kernels

$$\text{Linear:} \quad K(\mathbf{a}, \mathbf{b}) = \mathbf{a}^T\mathbf{b}$$

$$\text{Polynomial:} \quad K(\mathbf{a}, \mathbf{b}) = \left(\gamma\mathbf{a}^T\mathbf{b} + r\right)^d$$

$$\text{Gaussian RBF:} \quad K(\mathbf{a}, \mathbf{b}) = \exp\left(-\gamma\| \mathbf{a} - \mathbf{b} \|^2\right)$$

$$\text{Sigmoid:} \quad K(\mathbf{a}, \mathbf{b}) = \tanh\left(\gamma\mathbf{a}^T\mathbf{b} + r\right)$$

Kernel Trick in Kernel-SVM

How does Kernel Trick makes calculations efficient? Let's see an example.

Equation 5-9. Kernel trick for a 2nd-degree polynomial mapping

$$\phi(\mathbf{a})^T \cdot \phi(\mathbf{b}) = \begin{pmatrix} a_1^2 \\ \sqrt{2}\, a_1 a_2 \\ a_2^2 \end{pmatrix}^T \cdot \begin{pmatrix} b_1^2 \\ \sqrt{2}\, b_1 b_2 \\ b_2^2 \end{pmatrix}$$

Kernel Trick in Kernel-SVM

How does Kernel Trick makes calculations efficient? Let's see an example.

Equation 5-9. Kernel trick for a 2^{nd}-degree polynomial mapping

$$\phi(\mathbf{a})^T \cdot \phi(\mathbf{b}) = \begin{pmatrix} a_1^2 \\ \sqrt{2}\, a_1 a_2 \\ a_2^2 \end{pmatrix}^T \cdot \begin{pmatrix} b_1^2 \\ \sqrt{2}\, b_1 b_2 \\ b_2^2 \end{pmatrix} = a_1^2 b_1^2 + 2 a_1 b_1 a_2 b_2 + a_2^2 b_2^2$$

Kernel Trick in Kernel-SVM

How does Kernel Trick makes calculations efficient? Let's see an example.

Equation 5-9. Kernel trick for a 2nd-degree polynomial mapping

$$\phi(\mathbf{a})^T \cdot \phi(\mathbf{b}) = \begin{pmatrix} a_1^2 \\ \sqrt{2}\, a_1 a_2 \\ a_2^2 \end{pmatrix}^T \cdot \begin{pmatrix} b_1^2 \\ \sqrt{2}\, b_1 b_2 \\ b_2^2 \end{pmatrix} = a_1^2 b_1^2 + 2 a_1 b_1 a_2 b_2 + a_2^2 b_2^2$$

$$= (a_1 b_1 + a_2 b_2)^2 = \left(\begin{pmatrix} a_1 \\ a_2 \end{pmatrix}^T \cdot \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} \right)^2 = \left(\mathbf{a}^T \cdot \mathbf{b} \right)^2$$

Kernel Trick in Kernel-SVM

How does Kernel Trick makes calculations efficient? Let's see an example.

Equation 5-9. Kernel trick for a 2nd-degree polynomial mapping

$$\phi(\mathbf{a})^T \cdot \phi(\mathbf{b}) = \begin{pmatrix} a_1^2 \\ \sqrt{2}\, a_1 a_2 \\ a_2^2 \end{pmatrix}^T \cdot \begin{pmatrix} b_1^2 \\ \sqrt{2}\, b_1 b_2 \\ b_2^2 \end{pmatrix} = a_1^2 b_1^2 + 2 a_1 b_1 a_2 b_2 + a_2^2 b_2^2$$

3D dot product

$$= (a_1 b_1 + a_2 b_2)^2 = \left(\begin{pmatrix} a_1 \\ a_2 \end{pmatrix}^T \cdot \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} \right)^2 = \left(\mathbf{a}^T \cdot \mathbf{b} \right)^2$$

Kernel Trick in Kernel-SVM

How does Kernel Trick makes calculations efficient? Let's see an example.

Equation 5-9. Kernel trick for a 2nd-degree polynomial mapping

$$\phi(\mathbf{a})^T \cdot \phi(\mathbf{b}) = \begin{pmatrix} a_1^2 \\ \sqrt{2}\,a_1 a_2 \\ a_2^2 \end{pmatrix}^T \cdot \begin{pmatrix} b_1^2 \\ \sqrt{2}\,b_1 b_2 \\ b_2^2 \end{pmatrix} = a_1^2 b_1^2 + 2a_1 b_1 a_2 b_2 + a_2^2 b_2^2$$

3D dot product

2D dot product

$$= (a_1 b_1 + a_2 b_2)^2 = \left(\begin{pmatrix} a_1 \\ a_2 \end{pmatrix}^T \cdot \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} \right)^2 = \left(\mathbf{a}^T \cdot \mathbf{b}\right)^2$$

Kernel Trick in Kernel-SVM

How does Kernel Trick makes calculations efficient? Let's see an example.

Equation 5-9. Kernel trick for a 2nd-degree polynomial mapping

$$\phi(\mathbf{a})^T \cdot \phi(\mathbf{b}) = \begin{pmatrix} a_1^2 \\ \sqrt{2}\,a_1 a_2 \\ a_2^2 \end{pmatrix}^T \cdot \begin{pmatrix} b_1^2 \\ \sqrt{2}\,b_1 b_2 \\ b_2^2 \end{pmatrix} = a_1^2 b_1^2 + 2a_1 b_1 a_2 b_2 + a_2^2 b_2^2$$

3D dot product

2D dot product

$$= (a_1 b_1 + a_2 b_2)^2 = \left(\begin{pmatrix} a_1 \\ a_2 \end{pmatrix}^T \cdot \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} \right)^2 = \left(\mathbf{a}^T \cdot \mathbf{b}\right)^2$$

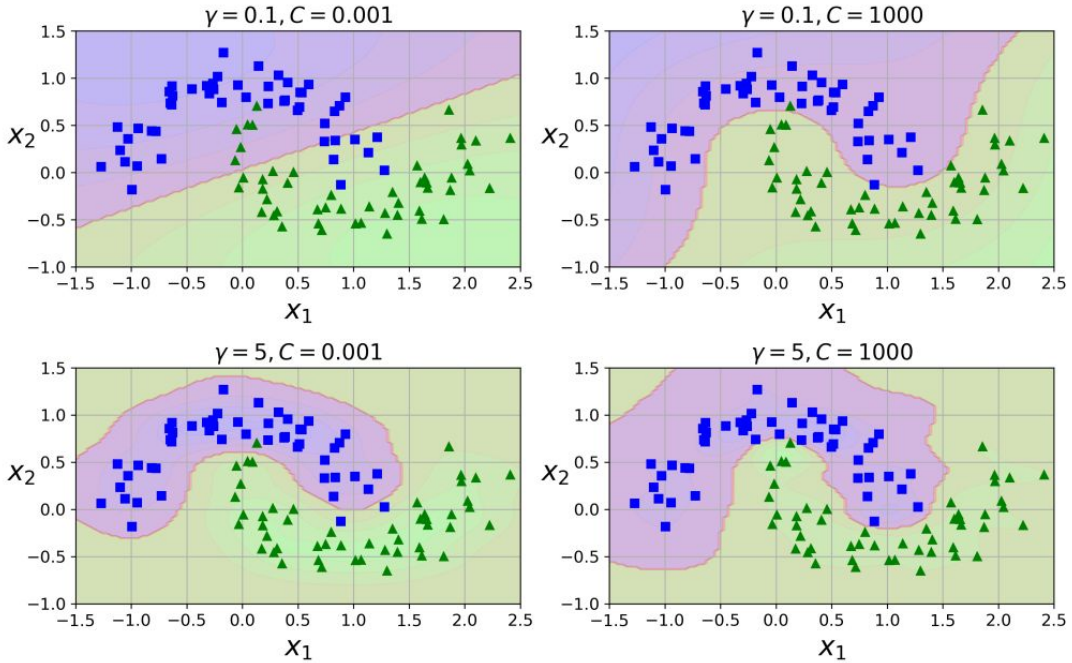
If ϕ is the 2nd -degree polynomial transformation, then you can replace this dot product of transformed vectors simply by the kernel. So you don't actually need to transform the training instances at all.

Kernel Trick in Kernel-SVM

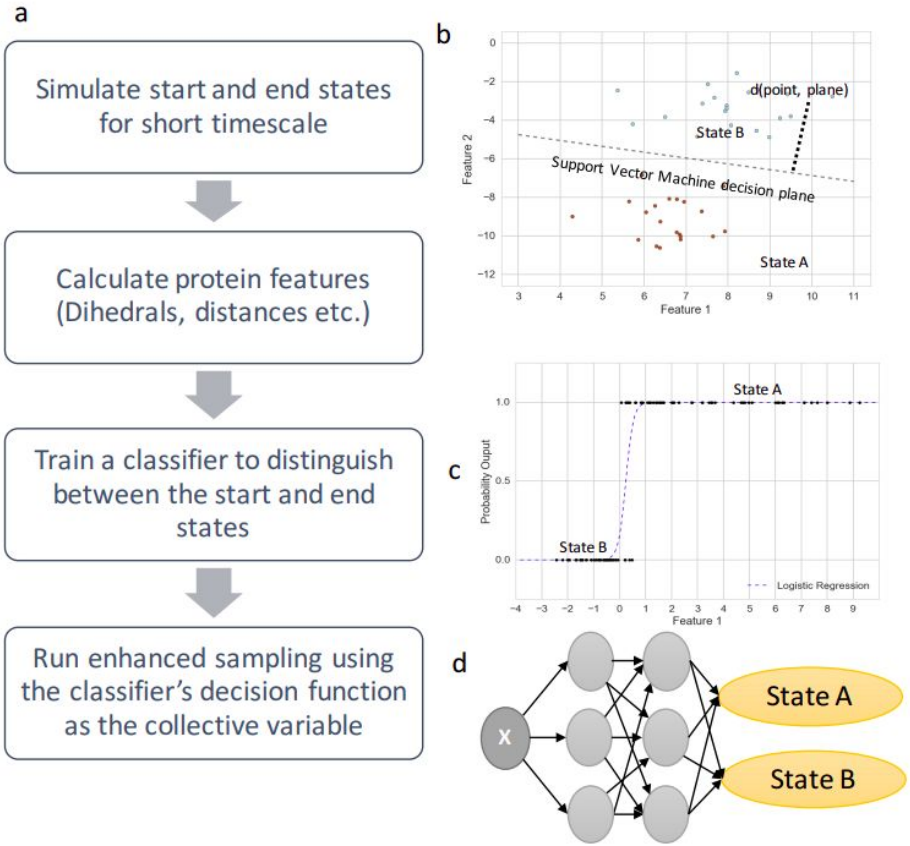
Most commonly used one is the Gaussian Kernel or Radial Basis Function (RBF)

$$\kappa\left(\boldsymbol{x}^{(i)}, \boldsymbol{x}^{(j)}\right) = \exp\left(-\gamma\left\|\boldsymbol{x}^{(i)} - \boldsymbol{x}^{(j)}\right\|^2\right)$$

The hyperparameter γ determines smoothness of the Gaussian Kernels.

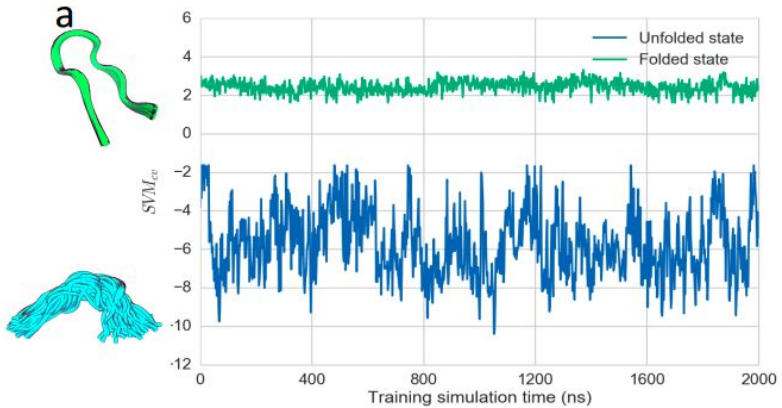


SVM and Logistic Regression Example



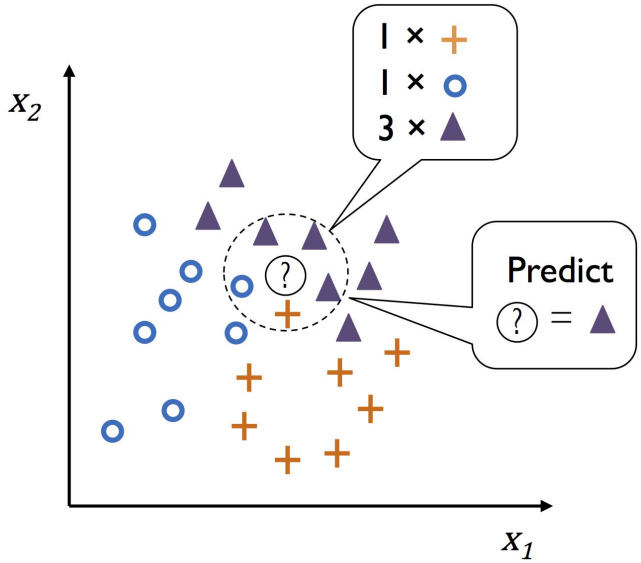
SVM and Logistic Regression for Protein Conformational State Classification.

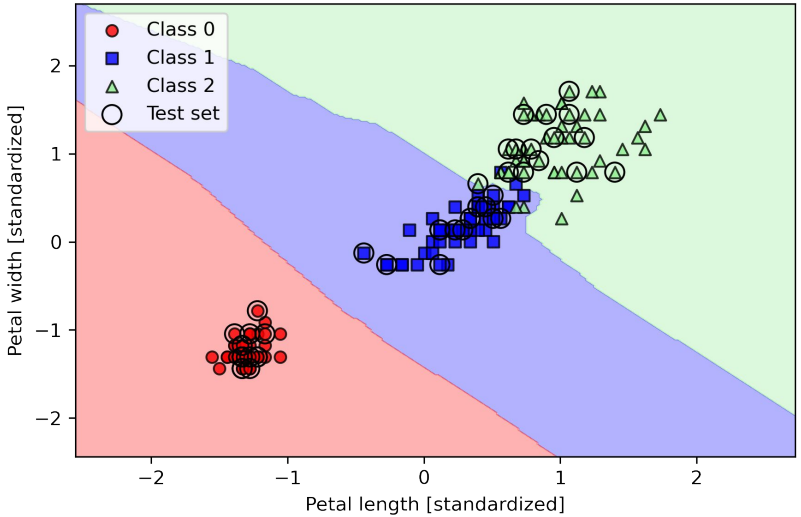
Sultan & Pandey. J. Chem. Phys. 149, 094106 (2018)
https://doi.org/10.1063/1.5029972



k-Nearest Neighbor (kNN) classification

1. Choose the integer k and a distance metric (e.g. Euclidean)
2. Find the k-nearest neighbors of the data record that we want to classify
3. Assign the class label by majority vote





k-Nearest Neighbor (kNN) classification

If Classification is so easy why did we learn so many complex methods?

kNN has a number of key limitations:

1. It doesn't learn a discriminative function from the training data but memorizes the training dataset instead.
2. Computational complexity for classifying new examples grows linearly with the number of examples in the training dataset.
3. Requires large memory to store the dataset.

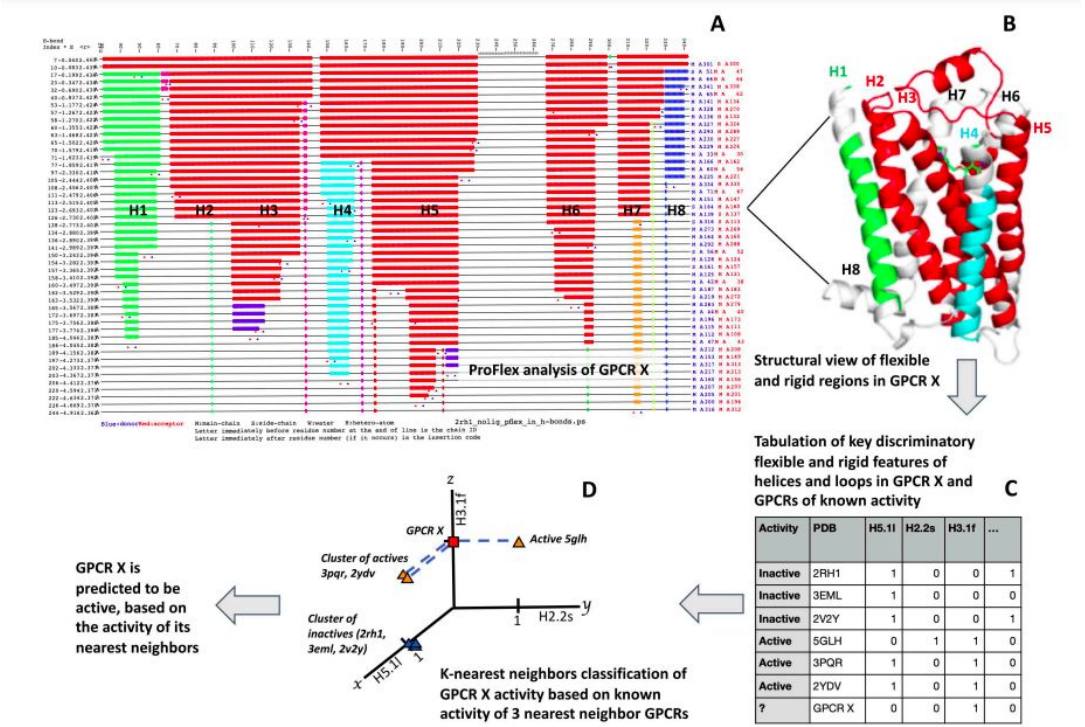
However, when we are working with relatively small to medium-sized data-sets, kNN can provide good predictive and computational performance.

Example: next slide

k-Nearest Neighbor (kNN) example

Machine Learning to Identify Flexibility Signatures of Class A GPCR Inhibition, *Biomolecules,* 2020, Joe Bemister-Buffington, Alex J. Wolf, **Sebastian Raschka***, and Leslie A. Kuhn, https://www.mdpi.com/2218-273X/10/3/454

*Author of the textbook we are following



Activity	PDB	H5.1l	H2.2s	H3.1f	...
Inactive	2RH1	1	0	0	1
Inactive	3EML	1	0	0	0
Inactive	2V2Y	1	0	0	1
Active	5GLH	0	1	1	0
Active	3PQR	1	0	1	0
Active	2YDV	1	0	1	0
?	GPCR X	0	0	1	0

Figure 2. Schematic of how: (**A**) ProFlex results for a GPCR correspond to (**B**) a three-dimensional structural representation of flexibility/rigidity; (**C**) these flexibility/rigidity features are tabulated as discrete features for machine learning; and (**D**) a KNN classifier is employed with the features for activity prediction.